

Laboratorio 03: Trabajo con Funciones JSON y T-SQL Avanzado en SQL Server

Título del laboratorio	Laboratorio 03: Trabajo con Funciones JSON y T-SQL Avanzado en SQL Server
Entorno de ejecución	Microsoft SQL Server 2025 / SSMS v20.x / Windows 11
Módulo	Advanced Data Processing & T-SQL
Base de datos	AdventureWorksLT
Alumno	Hugo Alcaide (alumno02)
Repositorio oficial	https://github.com/HugoAlcaide/master-labs
Fecha de realización	23 de septiembre de 2026

Objetivos del Laboratorio

1. Ejecutar operaciones analíticas y transformaciones directamente en el motor relacional mediante sintaxis basada en conjuntos (*set-based operations*).
2. Construir salidas JSON planas y estructuradas a partir de tablas relacionales mediante `FOR JSON PATH` y `JSON_OBJECT`.
3. Segmentar y clasificar catálogos de datos combinando Common Table Expressions (CTEs) y funciones de ventana (`ROW_NUMBER()` con `PARTITION BY`).
4. Ingestar y tipar colecciones semiestructuradas en memoria con `OPENJSON` y cruzarlas con tablas físicas mediante `INNER JOIN`.

Desarrollo Práctico y Evidencias de Ejecución

1. Verificación de Conectividad y Acceso al Esquema

Comprobación inicial de lectura sobre las entidades principales del catálogo de AdventureWorksLT.

```
-- Verify key tables in AdventureWorksLT
SELECT TOP (5) ProductID, Name, ListPrice
FROM SalesLT.Product;

SELECT TOP (5) ProductCategoryID, Name
FROM SalesLT.ProductCategory;
```

100 % ▼ ● No se encontraron problemas. Línea: 6, Carácter: 30 TABULACIONES CRLF UTF-8

Resultados Mensajes

	ProductID	Name	ListPrice
1	680	HL Road Frame - Black, 58	1431.50
2	706	HL Road Frame - Red, 58	1431.50
3	707	Sport-100 Helmet, Red	34.99
4	708	Sport-100 Helmet, Black	34.99
5	709	Mountain Bike Socks, M	9.50

	ProductCategoryID	Name
1	4	Accessories
2	22	Bib-Shorts
3	30	Bike Racks
4	31	Bike Stands
5	1	Bikes

● Consulta ejecutada correctamente. localhost (17.0 RTM) sa (79) AdventureWorksLT 00:00:00 Fila: 1, Columna: 1 10 filas

Evidencia 01: lectura correcta de las 5 primeras filas de productos y categorías en SSMS.

2. Conversión a JSON Plano con `FOR JSON PATH`

Transformación de filas relacionales en un array JSON continuo de objetos, delegando la serialización en el motor relacional.

```
SELECT
    ProductID,
    Name,
    Color,
    ListPrice
FROM SalesLT.Product
WHERE Color IS NOT NULL
ORDER BY ListPrice DESC
FOR JSON PATH;
```

100 % ▼ ● No se encontraron problemas. Línea: 9, Carácter: 15 SPC CRLF UTF-8

Resultados Mensajes

JSON_F52E2B61-18A1-11d1-B105-00805F49916B

1	[{"ProductID":749,"Name":"Road-150 Red, 62","Color":"Red","ListPrice":3578.2700...}]
---	--

● Consulta ejecutada correctamente. localhost (17.0 RTM) sa (79) AdventureWorksLT 00:00:00 Fila: 1, Columna: 1 1 filas

Evidencia 02: array JSON generado a partir de productos con atributo de color no nulo.

3. Estructuración Jerárquica con `JSON_OBJECT`

Construcción de un objeto anidado (`Category`) que agrupa el identificador y el nombre de la categoría dentro de cada elemento de producto.

```
SELECT
    p.ProductID,
    p.Name AS ProductName,
    p.ListPrice,
    JSON_OBJECT(
        'CategoryID': pc.ProductCategoryID,
        'CategoryName': pc.Name
    ) AS Category
FROM SalesLT.Product AS p
INNER JOIN SalesLT.ProductCategory AS pc
    ON p.ProductCategoryID = pc.ProductCategoryID
ORDER BY p.ListPrice DESC
FOR JSON PATH;
```

100 % ● No se encontraron problemas. Línea: 13, Carácter: 16 SPC CRLF UTF-8

Resultados		Mensajes	
JSON_F52E2B61-18A1-11d1-B105-00805F49916B			
1	[{"ProductID":749,"ProductName":"Road-150 Red, 62","ListPrice":3578.2700,"Category":{"CategoryID":1,"CategoryName":"Road Bikes"}}		

✓ Consulta ejecutada correctamente. localhost (17.0 RTM) sa (79) AdventureWorksLT 00:00:00 Fila: 1, Columna: 1 1 filas

Evidencia 03: salida JSON con la subestructura anidada Category integrada por fila.

4. Ranking por Categoría mediante CTE y ROW_NUMBER()

Uso de un CTE no recursivo y la función de ventana ROW_NUMBER() con particionado analítico para filtrar el Top 3 de productos con mayor precio por cada categoría.

```
WITH RankedProducts AS (  
    SELECT  
        p.ProductID,  
        p.Name AS ProductName,  
        pc.Name AS CategoryName,  
        p.ListPrice,  
        ROW_NUMBER() OVER (  
            PARTITION BY pc.ProductCategoryID  
            ORDER BY p.ListPrice DESC  
        ) AS PriceRank  
    FROM SalesLT.Product AS p  
    INNER JOIN SalesLT.ProductCategory AS pc  
        ON p.ProductCategoryID = pc.ProductCategoryID  
    WHERE p.ListPrice > 0  
)  
SELECT  
    ProductID,  
    ProductName,  
    CategoryName,  
    ListPrice,  
    PriceRank  
FROM RankedProducts  
WHERE PriceRank <= 3  
ORDER BY CategoryName, PriceRank;
```

100 % No se encontraron problemas. Línea: 24, Carácter: 34 SPC CRLF UTF-8

	ProductID	ProductName	CategoryName	ListPrice	PriceRank
1	855	Men's Bib-Shorts, S	Bib-Shorts	89.99	1
2	856	Men's Bib-Shorts, M	Bib-Shorts	89.99	2
3	857	Men's Bib-Shorts, L	Bib-Shorts	89.99	3
4	876	Hitch Rack - 4-Bike	Bike Racks	120.00	1
5	879	All-Purpose Bike Stand	Bike Stands	159.00	1
6	871	Mountain Bottle Cage	Bottles and Cages	9.99	1
7	872	Road Bottle Cage	Bottles and Cages	8.99	2
8	870	Water Bottle - 30 oz.	Bottles and Cages	4.99	3
9	996	HL Bottom Bracket	Bottom Brackets	121.49	1
10	995	ML Bottom Bracket	Bottom Brackets	101.24	2
11	994	LL Bottom Bracket	Bottom Brackets	53.99	3
12	948	Front Brakes	Brakes	106.50	1
13	907	Rear Brakes	Brakes	106.50	2
14	712	AWC Logo Cap	Caps	8.99	1
15	952	Chain	Chains	20.24	1
16	877	Bike Wash - Dissolver	Cleaners	7.95	1
17	951	HL Crankset	Cranksets	404.99	1
18	950	ML Crankset	Cranksets	256.49	2

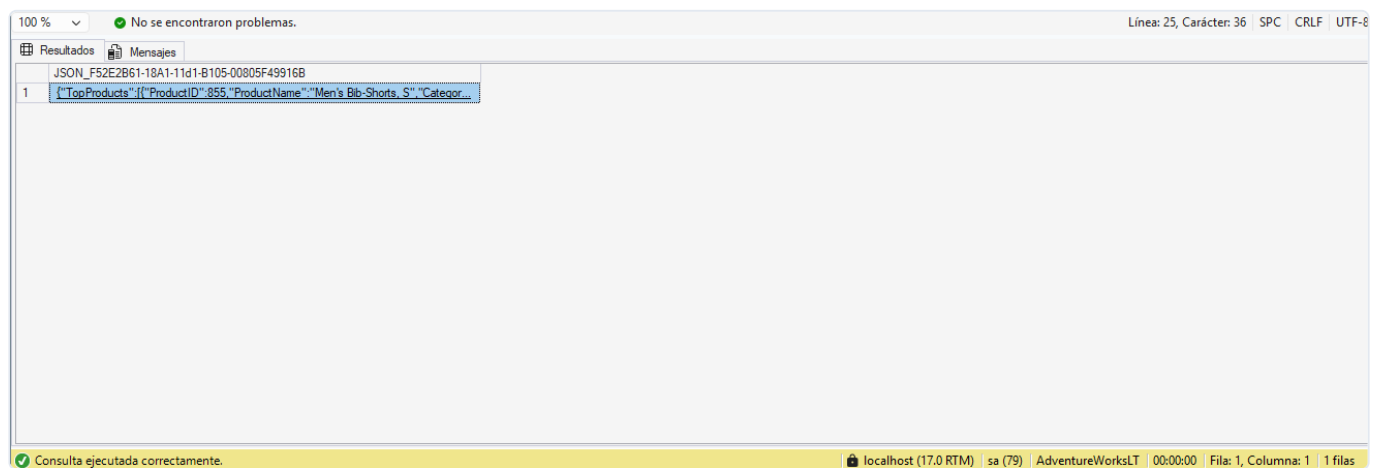
Consulta ejecutada correctamente. localhost (17.0 RTM) sa (79) AdventureWorksLT 00:00:00 Fila: 1, Columna: 1 90 filas

Evidencia 04: cuadrícula relacional filtrando exactamente los tres productos más caros de cada categoría.

5. Exportación del Ranking a JSON con Elemento Raíz (ROOT)

Envoltura del resultado analítico dentro de la clave raíz `"TopProducts"` para cumplir con especificaciones estándar de consumo en APIs REST.

```
WITH RankedProducts AS (  
    SELECT  
        p.ProductID,  
        p.Name AS ProductName,  
        pc.Name AS CategoryName,  
        p.ListPrice,  
        ROW_NUMBER() OVER (  
            PARTITION BY pc.ProductCategoryID  
            ORDER BY p.ListPrice DESC  
        ) AS PriceRank  
    FROM SalesLT.Product AS p  
    INNER JOIN SalesLT.ProductCategory AS pc  
        ON p.ProductCategoryID = pc.ProductCategoryID  
    WHERE p.ListPrice > 0  
)  
SELECT  
    ProductID,  
    ProductName,  
    CategoryName,  
    ListPrice,  
    PriceRank  
FROM RankedProducts  
WHERE PriceRank <= 3  
ORDER BY CategoryName, PriceRank  
FOR JSON PATH, ROOT('TopProducts');
```



Evidencia 05: payload JSON estructurado bajo la propiedad envoltente `TopProducts`.

6. Parseo de JSON a Conjunto Relacional con OPENJSON

Conversión de un documento JSON recibido en memoria a formato tabular mediante la cláusula `WITH` con tipos de datos relacionales explícitos (`INT`, `DECIMAL`).

```
DECLARE @ProductUpdates NVARCHAR(MAX) = N'[
  {"ProductID": 680, "NewPrice": 1250.00},
  {"ProductID": 706, "NewPrice": 1450.00},
  {"ProductID": 707, "NewPrice": 38.99}
]';

SELECT
    ProductID,
    NewPrice
FROM OPENJSON(@ProductUpdates)
WITH (
    ProductID INT '$.ProductID',
    NewPrice DECIMAL(10,2) '$.NewPrice'
);
```

	Resultados	Mensajes
	ProductID	NewPrice
1	680	1250.00
2	706	1450.00
3	707	38.99

Consulta ejecutada correctamente.

localhost (17.0 RTM) | sa (79) | AdventureWorksLT | 00:00:00 | Fila: 1, Columna: 1 | 3 filas

Evidencia 06: conjunto relacional derivado de JSON con columnas fuertemente tipadas.

7. Integración de OPENJSON con Tablas Relacionales (INNER JOIN)

Cruce entre la tabla virtual generada por OPENJSON y la tabla física SalesLT.Product para comparar precios vigentes frente a nuevos precios y calcular la diferencia directamente en SQL Server.

```
DECLARE @ProductUpdates NVARCHAR(MAX) = N'[
  {"ProductID": 680, "NewPrice": 1250.00},
  {"ProductID": 706, "NewPrice": 1450.00},
  {"ProductID": 707, "NewPrice": 38.99}
]';

SELECT
  p.ProductID,
  p.Name,
  p.ListPrice AS CurrentPrice,
  updates.NewPrice,
  updates.NewPrice - p.ListPrice AS PriceDifference
FROM SalesLT.Product AS p
INNER JOIN OPENJSON(@ProductUpdates)
WITH (
  ProductID INT '$.ProductID',
  NewPrice DECIMAL(10,2) '$.NewPrice'
) AS updates
ON p.ProductID = updates.ProductID;
```

100 %

No se encontraron problemas.

Línea: 19, Carácter: 40 SPC CRLF UTF-8

Resultados

Mensajes

	ProductID	Name	CurrentPrice	NewPrice	PriceDifference
1	680	HL Road Frame - Black, 58	1431.50	1250.00	-181.5000
2	706	HL Road Frame - Red, 58	1431.50	1450.00	18.5000
3	707	Sport-100 Helmet, Red	34.99	38.99	4.0000

Consulta ejecutada correctamente.

localhost (17.0 RTM) sa (79) AdventureWorksLT 00:00:00 Fila: 1, Columna: 1 3 filas

Evidencia 07: cruce relacional comparando ListPrice y NewPrice con el cálculo dinámico de PriceDifference.

Conclusiones Técnicas

- **Eficiencia Set-Based:** El cálculo de rankings dentro de particiones (ROW_NUMBER() OVER(PARTITION BY ...)) resuelve analíticas complejas en una única pasada sobre los datos, evitando el procesamiento iterativo en la aplicación.
- **Flexibilidad Semiestructurada:** El uso combinado de FOR JSON PATH y OPENJSON ... WITH dota a SQL Server de capacidades para operar de forma híbrida entre modelos relacionales y documentos JSON sin penalizaciones de mapeo ORM.